

NEXUSTEAM

Software Engineering Documentation

Final Project Documentation

Requirements • Architecture • Implementation • UI • Testing • Project Management •
Documentation

Team responsibilities

Sofiia Khyzhnychenko — Requirements Management, Architecture, Documentation

Vladyslav Zaplitniy — Backend and Client Implementation

Anna Kornet — User Interface

Halil Hakan Karabay — Testing

TABLE OF CONTENTS

1. Project Overview — ALL
2. Requirements Management — Sofiia Khyzhnychenko
3. Architecture — Sofiia Khyzhnychenko
4. Backend and Client Implementation — Vladyslav Zaplitniy
5. User Interface — Anna Kornet
6. Testing — Halil Hakan Karabay
7. Project Management — ALL
8. Documentation — ALL
9. Conclusion and Future Work — ALL

Appendix A — Diagrams

1. Project Overview

1.1 Introduction

NexusTeam is a multi-platform communication system designed for real-time digital communication. The project combines messaging with file sharing, user presence, chat organization, voice communication, image generation, and multiple client applications. The responsive web client is the universal browser-based client; the web application is an additional Windows web client.

The system is implemented as a client-server application with a browser-based web client. The web client is implemented using HTML, CSS, and JavaScript and communicates with the ASP.NET Core backend through REST APIs and WebSockets.

The main purpose of NexusTeam is to provide reliable real-time communication while maintaining a modular and scalable software architecture. The project was designed not only as a functional messaging application but also as a demonstration of modern software engineering practices, including layered architecture, separation of concerns, database specialization, containerization, authentication, real-time communication, and structured project management.

The backend of the system is implemented using ASP.NET Core 8. It acts as the central component responsible for authentication, user management, chat management, messaging, file attachments, user preferences, and real-time communication. The system uses several specialized infrastructure components, including Oracle Database, MongoDB, and Redis. Docker is used to simplify deployment and provide a reproducible development environment.

1.2 Motivation

Modern communication platforms require more than simple text messaging. Users expect messages to be delivered in real time, files to be shared conveniently, online status to be visible, and conversations to remain available across different devices and platforms. At the same time, communication systems need to address security, reliability, scalability, and maintainability.

The motivation behind NexusTeam was therefore to develop a complete communication platform that combines these requirements in a single system. The project also provides an opportunity to apply theoretical software engineering concepts in a practical environment.

A particular focus was placed on the separation of application responsibilities. Instead of storing all information in a single database or implementing all functionality in one application layer, NexusTeam separates user-related relational data, communication-related document data, and temporary or high-frequency data.

This approach allows different technologies to be used according to their specific strengths:

Oracle Database is used for structured user-related data.

MongoDB is used for chats, messages, attachment metadata, and other document-oriented data. The actual attachment files are stored by AttachmentService in the server-side file system.

Redis is used for fast-access and temporary data such as sessions, caching, and realtime-related information.

ASP.NET Core 8 provides the central backend application.

WebSockets provide realtime bidirectional communication.

web client provides a browser-based client.

1.3 Project Goals

The main goal of the project is to develop a functional and extensible messaging platform capable of supporting both traditional and real-time communication.

The primary objectives are:

To provide secure user registration and authentication.

To enable private and group-based communication.

To provide real-time message exchange.

To support editing and deletion of messages.

To support file and attachment sharing.

To provide user presence and status information.

To allow users to organize conversations using locally stored chat folders.

To support voice communication, with the current voice-message functionality provided by the web client.

1.4 Target Users

NexusTeam is primarily intended for users who require a convenient platform for direct and group communication.

The main target users include:

Individual Users

Individual users can register an account, authenticate themselves, communicate with other users, exchange messages, share files, and manage their personal settings.

Group Users

Users can participate in group conversations and interact with multiple participants within a shared chat environment.

Desktop Users

Windows users can access the system through the native web client. The web application provides access to the main communication functionality while integrating with the operating system more closely than a browser-based application.

Web Users

The responsive web client is the universal client and can be accessed from desktop and mobile browsers without installing the web application. The web client remains available as a Windows-only development client.

1.5 Main Features

NexusTeam contains a range of communication and supporting features.

User Authentication

The system provides account registration and authentication. Authentication is based on JWT tokens and refresh tokens, while passwords are protected using password hashing.

Authentication is implemented centrally on the backend so that both the desktop and web clients can use the same identity and authorization mechanisms.

Real-Time Messaging

Real-time messaging is one of the central features of NexusTeam. WebSocket communication is used to establish a persistent bidirectional communication channel between the client and the server.

This allows the system to transmit events such as:

- new messages;

- typing indicators;

- message delivery information;

1.6 Technology Stack

The project uses several technologies, each serving a specific purpose within the overall system.

The combination of these technologies reflects the project's requirement for multiple communication channels, different types of persistent data, and a modular architecture.

1.7 System Overview

At a high level, NexusTeam follows a client-server architecture. The clients are responsible for presentation and user interaction, while the backend provides centralized business logic, authentication, data access, and realtime communication.

The two main clients are:

- the NexusTeam Client, implemented as a browser-based web application;

- the NexusTeam Web Client, implemented using web technologies.

Both clients communicate with the same backend system. REST APIs are primarily used for request-response operations, while WebSockets are used for realtime events.

The backend is divided into multiple logical layers. Controllers receive requests, services contain business logic, repositories provide data access, and database systems store the corresponding information.

The resulting high-level structure can be summarized as follows:

Clients → REST/WebSocket → ASP.NET Core Server → Services/Repositories → Databases and Infrastructure

This structure reduces direct dependencies between the user interface and the databases. Clients do not directly access Oracle, MongoDB, or Redis. Instead, all data operations are mediated by the backend.

1.8 Project Structure

The solution is divided into several major projects and components.

NexusTeam.Server

The server contains the ASP.NET Core backend. It includes controllers, services, repositories, middleware, configuration, validation, authentication, and WebSocket handling.

`NexusTeam.Client`

The client contains the browser-based web application. It follows the modular web UI architecture pattern and separates views, view models, services, and infrastructure components.

`NexusTeam.Shared`

The Shared project contains models, DTOs, enums, exceptions, and communication contracts shared between different components of the system.

This reduces duplication and ensures that the client and server use consistent data structures and WebSocket contracts.

`NexusTeam.DbSeeder`

The database seeder is responsible for initializing the required database structures and development data.

1.9 Project Scope

The scope of the project includes the development of the communication platform, its clients, backend infrastructure, databases, security mechanisms, deployment environment, and supporting documentation.

The project covers both the functional and technical aspects of a modern messenger. This includes not only the user-facing interface but also the backend services, persistence layer, realtime communication infrastructure, authentication, and deployment configuration.

At the same time, the project should be considered within the scope of an academic software engineering project. Some components may require further development before being considered production-ready at the scale of a commercial messaging platform. Potential improvements include additional automated tests, stronger production secret management, further scalability testing, and additional monitoring.

1.10 Document Structure

This documentation describes the project from both a software engineering and implementation perspective.

The following chapters are organized according to the main aspects required for the project documentation:

Chapter 1 – Project Overview introduces NexusTeam, its motivation, goals, target users, main features, technology stack, and scope.

Chapter 2 – Requirements Management describes the functional and non-functional requirements and explains how they influenced the development process.

Chapter 3 – Architecture presents the overall system architecture, application layers, communication mechanisms, databases, infrastructure, and security architecture.

Chapter 4 – Backend and Client Implementation describes how the main backend and client features were implemented.

Chapter 5 – User Interface presents the design and implementation of the browser-based interface.

Chapter 6 – Testing describes the testing strategy, test cases, results, and identified limitations.

Chapter 7 – Project Management discusses team organization, task distribution, development workflow, version control, and project coordination.

Chapter 8 – Documentation describes the documentation created for developers, deployment, architecture, security, and maintenance.

2. Requirements Management

2.1 Introduction

Requirements management is an essential part of the NexusTeam development process. It defines what the system should provide to its users, which technical and quality characteristics the system should have, and which constraints have to be considered during development.

For a communication platform such as NexusTeam, requirements cover several different areas. The system has to provide basic messaging functionality, but it also has to support authentication, real-time communication, file sharing, user presence, multiple clients, data persistence, security, and reliable operation.

The requirements were therefore considered from both the functional and non-functional perspectives. Functional requirements describe what the system should do, while non-functional requirements describe how the system should perform these functions and which quality characteristics it should provide.

Requirements management also served as a connection between the project goals and the technical architecture. The architectural decisions described in Chapter 3 were influenced by the requirements identified in this chapter. For example, the requirement for real-time communication resulted in the use of WebSockets, while the requirement to support different types of persistent data contributed to the use of Oracle Database and MongoDB.

2.2 Project Requirements and Scope

The primary requirement of NexusTeam is to provide a centralized communication platform that allows registered users to communicate through different client applications.

The system should allow users to create accounts, authenticate themselves, find other users, create conversations, exchange messages, and manage their conversations. In addition to basic messaging, the platform should provide functionality for attachments, user status, chat organization, and voice communication.

The project also has a technical requirement to support more than one type of client. Therefore, the backend must provide a common interface that can be consumed by both the browser-based web application and the web client.

The system scope can be divided into five major areas:

- User management

- Communication and chat management

2.3 Stakeholders

A stakeholder is a person or group that has an interest in the system or can be affected by its development and operation.

The main stakeholders of NexusTeam are the following.

2.3.1 End Users

End users are the primary stakeholders. They interact with the system through the responsive web client or the browser-based web client. Both clients support the core

communication workflows, while the current voice-message functionality is provided by the web client.

Their main expectations are:

- simple interaction;
- reliable message delivery;
- fast response times;
- secure authentication;

2.3.2 Development Team

The development team is responsible for implementing, testing, documenting, and maintaining the system.

The team consists of members with different responsibilities:

The separation of responsibilities allows different technical areas to be developed in parallel while maintaining a common system architecture.

2.3.3 Project Supervisors / Evaluators

As an academic software engineering project, the system is also evaluated by supervisors or instructors. Their requirements include a clear software architecture, appropriate documentation, evidence of testing, understandable project management, and a working implementation.

2.3.4 System Administrators / Developers

Although NexusTeam is primarily designed for end users, developers and administrators are also relevant stakeholders. They need to be able to install, configure, monitor, debug, and maintain the system.

For this reason, installation, architecture, security, server, and client documentation are important parts of the project.

2.4 Functional Requirements

Functional requirements define the operations that NexusTeam must provide.

The requirements are identified using IDs so that they can be referenced in later chapters.

2.4.1 User Registration

FR-01 — User Registration

The system shall allow a new user to create an account by providing the required registration information.

The server is responsible for validating the registration request and creating the corresponding user record.

The registration process must prevent the creation of invalid or duplicate accounts according to the validation rules implemented by the backend.

2.4.2 User Authentication

FR-02 — User Login

The system shall allow registered users to authenticate using their account credentials.

After successful authentication, the server provides the required authentication information to the client.

Authentication is implemented centrally so that the desktop and web clients use the same backend authentication mechanism.

2.4.3 Session Management

FR-03 — Session and Token Management

The system shall maintain authenticated sessions using access tokens and refresh tokens.

The system shall also support session-related operations such as logout and token invalidation.

This requirement is important because a messaging application has to maintain authenticated communication while minimizing the need for users to repeatedly enter their credentials.

2.4.4 User Profiles

FR-04 — User Profile Management

The system shall allow users to access and manage their profile-related information.

Profile functionality includes user information and avatar-related functionality.

The system should ensure that profile data is accessed only through authorized operations.

2.4.5 User Search and Directory

FR-05 — User Search

The system shall provide functionality for finding users who can participate in conversations.

This functionality is required because users need a mechanism for identifying other accounts before starting direct communication.

2.5 Chat Requirements

2.5.1 Chat Creation

FR-06 — Create Chat

The system shall allow users to create conversations.

Depending on the supported chat type, a conversation may contain an individual user or multiple participants.

The server is responsible for validating the creation request and storing the corresponding chat information.

2.5.2 Chat Management

FR-07 — Manage Chat

The system shall provide operations for managing existing conversations.

These operations include retrieving chat information and managing participants or other chat-related properties where applicable.

Only authorized users should be able to perform operations affecting a conversation.

2.5.3 Chat Folders

FR-08 — Chat Folders

The system shall allow users to organize conversations into chat folders.

Folders provide an additional organizational layer and allow users to group conversations according to their own preferences.

The folder functionality is implemented as a separate backend component and is also represented in the client application.

2.6 Messaging Requirements

2.6.1 Sending Messages

FR-09 — Send Message

The system shall allow authenticated users to send messages to conversations in which they participate.

A message must contain the necessary information to identify:

- the sender;
- the conversation;
- the message content;

2.6.2 Real-Time Message Delivery

FR-10 — Real-Time Delivery

Messages should be delivered to connected participants in real time.

This requirement is one of the key requirements of NexusTeam and directly influences the architecture. A conventional request-response mechanism alone would not provide the desired communication experience. Therefore, the system uses WebSocket connections for realtime events.

2.6.3 Message Editing

FR-11 — Edit Message

The system shall allow an authorized user to modify a previously sent message.

The system must ensure that users cannot arbitrarily modify messages belonging to other users.

Changes must be synchronized with other connected clients.

2.6.4 Message Deletion

FR-12 — Delete Message

The system shall provide functionality for deleting messages.

Message deletion must be controlled by authorization rules and reflected in the state presented to connected clients.

2.6.5 Message Status

FR-13 — Message Status

The system shall support message-related status information.

The realtime communication layer includes contracts for events such as message delivery and message reading.

This allows clients to synchronize message state between participants.

2.7 Attachment Requirements

2.7.1 File Upload

FR-14 — File Attachments

The system shall allow users to attach files to messages.

The backend provides dedicated attachment functionality for handling uploaded files and associated metadata.

2.7.2 Attachment Access

FR-15 — Attachment Retrieval

Authorized users shall be able to access attachments associated with conversations in which they participate.

Attachment operations must respect the same authorization principles as message operations.

2.8 Realtime Communication Requirements

Realtime functionality is a major part of the NexusTeam system.

2.8.1 Persistent WebSocket Connection

FR-16 — WebSocket Communication

The client shall be able to establish an authenticated WebSocket connection with the server.

The WebSocket connection is used for bidirectional communication between clients and the backend.

2.8.2 Typing Indicators

FR-17 — Typing Indicators

The system shall support realtime typing indicators.

When a user starts or stops typing, the corresponding event can be transmitted to other participants in the conversation.

2.8.3 User Presence

FR-18 — User Presence

The system shall support realtime user status and presence information.

Status changes can be transmitted through dedicated WebSocket contracts so that connected clients can update the corresponding UI.

2.8.4 Chat Events

FR-19 — Chat Events

The realtime subsystem shall support events related to users joining or leaving conversations and other relevant chat state changes.

This allows connected clients to maintain a synchronized view of the conversation.

2.9 Voice Communication Requirements

The system shall allow a user to initiate a voice communication session with another participant.

The voice-message subsystem provides dedicated signalling contracts for call requests and responses.

FR-21 — Call Control

The system shall support the main call lifecycle operations, including:

2.10 Additional Functional Requirements

2.10.1 User Preferences

FR-22 — User Preferences

The system shall allow users to store and retrieve personal preferences.

The preferences subsystem is implemented separately from the core messaging functionality.

2.10.2 Image Generation

FR-23 — Generated Images

The system shall provide functionality for working with generated images.

Generated-image operations are represented by a dedicated backend service, controller, repository, and corresponding client functionality.

2.10.3 Translation

FR-24 — Translation Support

The client contains translation-related functionality that allows translated content to be presented to the user.

This feature extends the communication capabilities of the platform and demonstrates the possibility of integrating additional services into the messenger.

2.10.4 Voice Messages

FR-25 — Voice Messages

The browser client shall allow users to record and send voice messages within conversations. Recipients shall be able to access and play the recorded audio.

2.10.5 Local Chat Folders

FR-26 — Local Chat Organization

The browser client shall allow users to create and manage local folders for organizing conversations. Folder organization is a client-side preference and does not represent a separate server-side conversation entity.

2.10.6 Interface Themes

FR-27 — Light and Dark Themes

The browser client shall allow users to switch between light and dark interface themes. The selected theme shall be stored as a local user preference and restored when the application is opened again.

2.11 Non-Functional Requirements

Functional requirements describe what NexusTeam does. However, a communication platform also needs to meet quality requirements.

The main non-functional requirements are divided into the following categories:

- security;

- performance;

- reliability;

- usability;

2.12 Security Requirements

Security is especially important because the system processes user accounts, authentication credentials, messages, and potentially private files.

NFR-01 — Secure Password Storage

Passwords shall not be stored as plaintext.

The backend uses BCrypt-based password hashing to protect stored credentials.

NFR-02 — Token-Based Authentication

Authenticated API operations shall be protected using token-based authentication.

2.13 Performance Requirements

NFR-09 — Realtime Responsiveness

Messages and other realtime events should be delivered with minimal delay when the client has an active connection.

This requirement is one of the reasons WebSocket communication is used instead of relying exclusively on polling.

NFR-10 — Efficient Data Access

The system shall use appropriate database technologies and indexes to support efficient access to frequently requested information.

MongoDB is used for document-oriented communication data, while Redis provides high-speed access for temporary and frequently accessed data.

2.14 Reliability Requirements

NFR-12 — Connection Recovery

The client should be able to recover from temporary WebSocket connection failures.

The client contains reconnection functionality designed to restore communication after connection interruptions.

NFR-13 — Offline Message Handling

The web client includes an offline message queue.

This provides a mechanism for handling messages that cannot immediately be transmitted because of temporary connectivity problems.

2.15 Usability Requirements

NFR-16 — Intuitive Navigation

The system should provide a clear navigation structure that allows users to move between conversations, settings, profiles, and additional functionality.

NFR-17 — Consistent User Experience

The main communication concepts should remain consistent across the desktop and web clients.

Although the clients have different technologies and interfaces, they communicate with the same backend and provide access to the same core system functionality.

NFR-18 — Feedback

2.16 Maintainability Requirements

NFR-19 — Separation of Responsibilities

The system should separate presentation, business logic, and data access.

The backend therefore uses controllers, services, and repositories as separate logical layers.

NFR-20 — Shared Contracts

Common models, DTOs, enums, and WebSocket contracts should be centralized where possible.

The NexusTeam.Shared project provides these shared definitions and reduces duplication between client and server components.

2.17 Scalability Requirements

NFR-22 — Database Scalability

The system should allow communication data to be stored using a database model suitable for document-oriented and potentially large datasets.

MongoDB is used for chats and messages, while the project infrastructure includes a MongoDB deployment with sharding-related components.

NFR-23 — Infrastructure Scalability

The system should be deployable using containerized components.

Docker allows infrastructure components to be separated and deployed independently.

2.18 Compatibility Requirements

NFR-25 — Desktop Compatibility

The native client shall support Windows environments compatible with the web application and its required .NET runtime.

NFR-26 — Web Compatibility

The web client shall be accessible through a modern web browser supporting the required HTML, CSS, and JavaScript functionality.

NFR-27 — Common Backend

Both client types shall use the same backend services rather than implementing separate business logic for each platform.

2.19 Deployment Requirements

NFR-28 — Containerized Deployment

The backend infrastructure shall be deployable using Docker.

The project includes Docker configuration for the server, web client, databases, Redis, and supporting infrastructure.

NFR-29 — Database Initialization

The project shall provide a mechanism for initializing the required databases and development data.

This responsibility is handled by the NexusTeam.DbSeeder project.

2.20 Requirements Prioritization

Not all requirements have the same importance. Requirements were therefore conceptually divided into three priority levels.

High Priority

These requirements are essential for the core system:

user registration;

authentication;

chat creation;

2.21 Requirements Traceability

Requirements traceability connects individual requirements to the components responsible for implementing them.

This makes it possible to determine where a requirement is implemented and where it should be tested.

This traceability approach is useful because it provides a direct relationship between requirements, implementation, architecture, and testing.

For example:

FR-10 — Real-Time Delivery

→ Architecture: WebSocket communication

2.22 Requirements and Architectural Decisions

Several important architectural decisions can be directly connected to the identified requirements.

The requirement for real-time messaging led to the adoption of WebSockets. Unlike a purely REST-based architecture, WebSockets allow the server to push events to connected clients without requiring continuous polling.

The requirement to support multiple types of data influenced the decision to use different database technologies. User-related structured data is stored in Oracle, while chat and message data is handled through MongoDB. Redis is used where fast access to temporary or frequently changing data is beneficial.

The requirement to support both desktop and web clients resulted in a centralized backend. Both clients use the same server APIs and communication protocols, which prevents the duplication of core business logic.

Security requirements influenced the authentication architecture. JWT, refresh tokens, password hashing, rate limiting, security headers, and WebSocket authentication are incorporated into the server architecture.

Maintainability requirements influenced the separation of the system into Server, Client, Shared, and DbSeeder projects.

2.23 Requirements Management Process

Requirements were managed throughout the development process rather than being considered only at the beginning of the project.

The general process consisted of the following stages:

Identification — determining the required functionality and quality characteristics.

Analysis — examining dependencies, technical constraints, and feasibility.

Prioritization — identifying which requirements were essential and which were additional.

Assignment — connecting requirements to project components and team responsibilities.

2.24 Requirements Constraints

The project was developed under several constraints.

Technical Constraints

The project had to integrate multiple technologies, including ASP.NET Core, web client, WebSocket, Oracle, MongoDB, Redis, and Docker.

Time Constraints

As an academic project, development was performed within a predefined project period. This required prioritization of core functionality over optional extensions.

Team Constraints

2.25 Requirements Risks

Several risks were considered during requirements management.

Scope Expansion

A messaging system can easily become very large because users may expect additional functionality such as video calls, reactions, encryption, notifications, synchronization, and integrations.

To reduce this risk, the project focused on a defined set of core and extended requirements.

Technical Complexity

The use of multiple databases and communication technologies increases the complexity of the system.

2.26 Summary

The requirements management process established the functional and quality objectives of NexusTeam and provided the foundation for the subsequent architecture and implementation decisions.

The most important functional requirements are secure authentication, chat management, realtime messaging, message management, attachment handling, user presence, and voice communication. Additional functionality such as chat folders, preferences, translation, and image generation extends the basic communication capabilities.

The non-functional requirements focus primarily on security, realtime performance, reliability, maintainability, usability, scalability, compatibility, and deployability.

A traceability relationship was established between requirements and system components. This allows requirements to be followed through the architecture, implementation, and testing phases.

The next chapter describes how these requirements were translated into the overall system architecture of NexusTeam.

3. Architecture

3.1 Introduction

The architecture of NexusTeam was designed to support the requirements described in the previous chapter while maintaining a clear separation of responsibilities between the different parts of the system.

NexusTeam follows a client-server architecture in which the backend acts as the central point of communication between clients and the persistence and infrastructure layers. Two different client applications are provided: a browser-based client based on Windows Presentation Foundation (web client) and a web client based on HTML, CSS, and JavaScript.

The backend is implemented using ASP.NET Core 8 and provides both REST-based HTTP endpoints and a WebSocket-based realtime communication channel. REST APIs are used for operations that follow a conventional request-response model, while WebSockets provide persistent bidirectional communication for realtime events.

The system also uses multiple infrastructure technologies. Oracle Database is used for structured user-related data, MongoDB is used for document-oriented communication data, and Redis is used for caching, session-related information, user status, and other high-frequency or temporary data.

The overall architecture can therefore be described as a combination of:

- client-server architecture;
- layered backend architecture;
- modular web UI architecture for the web client;

3.2 Architectural Goals

The architecture was designed around several major goals.

Separation of Responsibilities

Different concerns should be implemented in separate components. User interface code should not directly access databases, while database-specific functionality should not be mixed with presentation logic.

Reusability

The same backend should support both the desktop and web clients. Shared DTOs and communication contracts should reduce duplication between components.

Maintainability

The system should be organized into components that can be modified independently. Controllers, services, repositories, clients, and infrastructure components therefore have distinct responsibilities.

Real-Time Communication

3.3 High-Level System Architecture

At the highest level, the system consists of client applications, an application server, and infrastructure components.

3.4 Client-Server Architecture

NexusTeam uses a centralized backend that provides services to multiple clients.

The clients are responsible primarily for:

- presentation;
- user interaction;
- local UI state;
- client-side services;
- communication with the server.

The backend is responsible for:

3.5 Backend Architecture

The backend is implemented as an ASP.NET Core 8 application.

The main backend components are organized into several logical layer

3.6 Controllers Layer

Controllers represent the HTTP API boundary of the backend.

The server contains controllers for the major functional areas of the application, including:

- authentication;
- users;
- chats;
- messages;
- attachments;
- preferences;

3.7 Service Layer

The service layer contains the main business logic of the application.

The server registers dedicated services for major functional areas, including:

- AuthService;
- MessageService;
- ChatService;
- ChatFolderService;
- AttachmentService;
- GeneratedImageService;

3.8 Repository Layer

The repository layer abstracts database access from the business logic.

The server defines repository interfaces such as:

- IUserRepository;
- IChatRepository;
- IMessageRepository;
- IMessageAttachmentRepository;
- IChatFolderRepository;
- IUserPreferenceRepository;

3.9 Polyglot Persistence Architecture

One of the major architectural decisions in NexusTeam is the use of multiple data storage technologies.

Instead of using a single database for every type of information, the project uses a polyglot persistence approach.

The three main infrastructure components are:

This decision reflects the different characteristics of the stored data.

3.10 Oracle Database

Oracle is used primarily for structured user-related information.

The server contains an `OracleDataContext`, which creates and manages Oracle database connections using the Oracle Managed Data Access provider.

The database access is abstracted through `IOracleDataContext`.

The general access flow is:

User-related Service



`IUserRepository`



3.11 MongoDB

MongoDB is used for document-oriented communication data.

The project contains dedicated MongoDB repository implementations for:

chats;

messages;

message attachments;

chat folders;

user preferences;

generated images.

3.12 Redis

Redis provides a separate infrastructure layer for data that benefits from fast access or does not necessarily need to be stored in the primary databases.

The server registers:

`IRedisMultiplexer`;

`ICacheService`;

`IUserStatusService`.

Redis is used in areas such as:

caching;

session-related state;

3.13 REST API Architecture

The REST API is used for conventional request-response operations.

Typical operations include:

registration;

login;

retrieving users;

- creating chats;
- retrieving chats;
- retrieving messages;

3.14 WebSocket Architecture

REST alone is not sufficient for a realtime messaging system because the client would otherwise need to repeatedly request updates from the server.

NexusTeam therefore implements a dedicated WebSocket communication layer.

The server enables WebSockets and routes requests to the /ws endpoint.

The communication flow is approximately:

Client



Websocket connection

3.15 WebSocket Message Envelope

Realtime communication is standardized using a shared WebSocket message envelope.

The shared project contains:

- WebSocketMessageEnvelope;

- WebSocketMessageType;

- dedicated contracts for different event types.

This allows different types of realtime messages to be transported through a common protocol.

Conceptually:

WebSocket Message

3.16 WebSocket Connection Management

Active connections are managed by a dedicated WebSocketConnectionManager.

This component provides an abstraction around currently connected users and their WebSocket connections.

This is important because a user may have different communication states depending on whether they are connected.

The connection manager can be used by backend services to send messages to a specific user without requiring those services to directly manipulate WebSocket objects.

The architecture therefore follows:

Application Service



IWebSocketConnectionManager

3.17 Shared Architecture

The NexusTeam.Shared project is an important architectural component.

It provides definitions that need to be consistent between the server and clients.

The project contains:

- models;

- DTOs;
- contracts;
- enums;
- exceptions;

3.18 Web Client Architecture

The current NexusTeam client is browser-based and is implemented using standard web technologies rather than a native desktop UI framework.

The client uses HTML for the document structure, CSS for presentation and responsive layout, and JavaScript for application logic, state management, browser interaction, and communication with the backend.

The main client responsibilities include authentication, chat navigation, message presentation, message composition, attachment handling, voice-message recording, local chat-folder organization, theme selection, notifications, settings, and image-generation interaction.

The simplified client structure is:

HTML



CSS



JavaScript



REST API / WebSocket



ASP.NET Core Backend

This structure keeps the browser presentation layer separate from backend business logic and persistence.

3.22 Offline and Reconnection Architecture

Realtime communication depends on network connectivity, which means connection failures must be considered.

The web client contains mechanisms for reconnection and offline message handling.

3.23 Web Client Architecture

The project also contains a browser-based client.

Unlike the web client, the web client does not use a large framework such as Angular or React. It is implemented using:

- HTML;

- CSS;

- JavaScript.

The web client communicates with the same backend as the web client.

Its simplified architecture is:

HTML

3.24 Authentication Architecture

Authentication is centralized in the backend.

The main authentication components include:

AuthService;

JwtTokenService;

RefreshTokenService;

SessionService;

BcryptPasswordHasher;

JwtAuthenticationMiddleware.

3.25 Middleware Architecture

The backend uses a custom middleware pipeline.

The server configures middleware for:

security headers;

exception handling;

request logging;

JWT authentication;

WebSocket handling.

Conceptually:

3.26 Validation Architecture

Input validation is implemented as a separate concern.

The server registers FluentValidation validators and automatically applies validation to incoming requests.

This means that invalid requests can be rejected before reaching deeper business logic.

The resulting flow is:

Request

↓

Validation

↓

3.27 Dependency Injection

Dependency Injection is used extensively throughout the backend.

Instead of services directly constructing their dependencies, interfaces are registered in the application's dependency injection container.

Examples include:

IUserRepository

IChatRepository

IMessageRepository

IAuthService

IMessageService

3.28 Docker and Deployment Architecture

Docker is used to provide a reproducible infrastructure environment.

The project contains Docker configuration for the major components of the system, including:

- NexusTeam Server;
- Web Client;
- Oracle;
- MongoDB;
- MongoDB configuration infrastructure;
- Redis;

3.29 Configuration Architecture

Configuration is separated from application logic.

The backend contains dedicated option classes for configuration areas such as:

- Oracle;
- MongoDB;
- Redis;
- JWT;
- BCrypt;
- CORS;

3.30 Security Architecture

Security is implemented across multiple layers rather than being concentrated in a single component.

The main security mechanisms include:

Authentication

JWT tokens and refresh tokens are used to establish authenticated sessions.

Password Protection

Passwords are processed using BCrypt hashing.

Authorization

Operations are restricted based on the authenticated user and the context of the requested resource.

3.31 Architectural Decision: REST and WebSocket Together

One of the key architectural decisions was to use both REST and WebSocket communication instead of attempting to use a single protocol for all operations.

REST is well suited for operations where the client explicitly requests a resource or performs an action.

Examples include:

- login;
- retrieving chat information;
- retrieving message history;
- updating settings;
- uploading files.

3.32 Architectural Decision: Multiple Databases

Using Oracle, MongoDB, and Redis increases infrastructure complexity compared with using a single database.

However, the decision provides a clear separation between different categories of data.

Structured account data



Oracle

Communication documents



MongoDB

3.33 Architectural Decision: Shared Project

The NexusTeam.Shared project was introduced to avoid duplicated definitions between server and client components.

Without a shared project, the server and clients would need to independently define DTOs and realtime contracts.

That would create a risk of incompatibility.

The shared project instead provides a common contract:

NexusTeam.Shared

/ \

/ \



3.34 Architectural Decision: modular web UI architecture

The web client uses modular web UI architecture to separate the visual layer from application state and operations.

This provides several advantages:

reduced coupling between UI and application logic;

clearer code organization;

reusable services;

easier maintenance;

better testability;

easier modification of UI components.

3.35 Architectural Strengths

The architecture provides several important strengths.

Clear Separation of Concerns

Controllers, services, repositories, clients, and infrastructure components have distinct responsibilities.

Multiple Client Support

The same backend supports both web client and web clients.

Realtime Communication

WebSockets provide the communication mechanism required for a modern messenger.

Database Specialization

3.36 Architectural Limitations

The architecture also introduces several areas that require careful maintenance.

The use of multiple databases increases operational complexity. Developers need to understand Oracle, MongoDB, and Redis, and deployment requires several infrastructure components.

The WebSocket subsystem also introduces additional complexity compared with a purely REST-based application. Connection lifecycle, authentication, reconnection, message routing, and connection state all have to be managed.

The system also contains multiple clients, which increases the integration surface. A change to a shared DTO or WebSocket contract can potentially affect the server, web client, and web client.

Finally, the use of containerized database infrastructure provides a strong development environment but requires additional configuration compared with a simple single-database application.

These trade-offs were accepted because they support the project's functional and architectural goals.

3.37 Architecture-to-Requirements Mapping

The relationship between the requirements and architecture can be summarized as follows:

This mapping demonstrates that the architecture was derived from the requirements rather than being independent of them.

3.38 Summary

NexusTeam implements a modular client-server architecture designed around realtime communication, multiple clients, data specialization, and maintainability.

The ASP.NET Core 8 backend acts as the central application layer. Controllers expose HTTP endpoints, services implement business logic, repositories abstract data access, and middleware handles cross-cutting concerns such as authentication, logging, security, exceptions, and WebSocket communication.

The browser-based web application follows the modular web UI architecture pattern, while the web client uses HTML, CSS, and JavaScript. Both clients communicate with the same backend through REST APIs and WebSockets.

The system uses Oracle for structured user data, MongoDB for communication-related documents, and Redis for caching and temporary state. Docker provides a reproducible environment for the application and its infrastructure.

The NexusTeam.Shared project provides common DTOs, models, enums, and realtime communication contracts, reducing duplication and maintaining consistency between components.

Overall, the architecture provides a clear foundation for the implementation described in the following chapter. The next chapter focuses on how the architectural components are implemented in the backend and client applications.

Figure 3.1 — Database Diagram

Figure 3.2 — Class Diagram

4. Backend and Client Implementation

4.1 Introduction

The implementation of NexusTeam follows the architectural principles described in Chapter 3. The system is divided into a server application, a web client, a shared project, a database seeding application, and a web client.

The backend is implemented using C# and ASP.NET Core 8. It provides the main business logic, authentication, data access, REST API, and realtime communication. The web client is implemented using web client and follows the modular web UI architecture pattern. The responsive web client is the universal browser-based interface and communicates with the same backend. The web application is an additional native Windows client.

The implementation is organized around several principles:

- separation of responsibilities;
- dependency injection;
- reusable services;

4.2 Backend Project Structure

The backend is implemented in the NexusTeam.Server project.

The main functional areas include:

- Controllers;
- Services;
- Repositories;
- Middleware;

4.3 Dependency Injection

Dependency Injection is used throughout the server implementation.

Instead of creating dependencies directly inside controllers and services, interfaces are registered with the ASP.NET Core dependency injection container.

Examples include:

- IAuthService;
- IChatService;
- IMessageService;

4.4 Authentication Implementation

Authentication is implemented through several dedicated components.

The main authentication components are:

- AuthController;
- AuthService;
- JwtTokenService;
- RefreshTokenService;

4.5 JWT and Refresh Tokens

The application uses JWT-based authentication for protected API operations.

The access token provides the information required to authenticate subsequent requests.

Refresh tokens are used to maintain longer-lived sessions without requiring the user to repeatedly enter their credentials.

The implementation separates token generation from authentication business logic.

AuthService handles the authentication workflow, while JwtTokenService is responsible for token creation.

This separation makes it possible to modify the token-generation mechanism without changing the complete authentication workflow.

4.6 Password Hashing

Passwords are protected using BCrypt.

The hashing functionality is encapsulated in BcryptPasswordHasher.

The authentication process can therefore be described as:

Registration



4.7 User Management

User-related functionality is implemented through a combination of controller, service, and repository components.

The main operations include:

- retrieving user information;

- searching for users;

- updating profile information;

- managing avatars;

4.8 Chat Implementation

Chat functionality is implemented through the ChatsController and corresponding chat services and repositories.

The main operations include:

- creating chats;

- retrieving chats;

- modifying chat information;

- managing participants;

4.9 Message Implementation

Messaging is one of the central parts of the implementation.

The backend provides a dedicated MessageService and message repository.

The message model contains information such as:

- message identifier;

- chat identifier;

- sender identifier;

4.10 Sending Messages

When a user sends a message, the client creates a request containing the relevant chat and message information.

The backend processes the request through the message service.

A simplified flow is:

User enters message



4.11 Message Editing and Deletion

The message service provides functionality for modifying existing messages.

Before modifying a message, the backend can verify that the requesting user has the required permissions.

The editing process follows the same layered structure as other message operations.

EditMessageRequest



4.12 Message History

The backend provides functionality for retrieving previously stored messages.

Message history is stored in MongoDB and retrieved through the message repository.

This allows the client to load conversation history independently of the realtime connection.

This distinction is important because WebSockets provide current events, while the database provides persistent historical information.

The two mechanisms therefore complement each other:

Historical messages

4.13 Attachment Implementation

File handling is separated into dedicated components.

The backend contains:

AttachmentsController;

AttachmentService;

MessageAttachmentRepository;

attachment-related DTOs and models.

4.14 Chat Folder Implementation

Chat folders are implemented as a separate feature instead of being embedded directly into the main chat service.

The server provides:

ChatFoldersController;

ChatFolderService;

IChatFolderRepository;

corresponding DTOs and models.

4.15 User Preferences

User preferences are handled by a dedicated preferences controller and service.

Preferences are separated from core user information because they represent configuration rather than account identity.

The implementation provides operations for retrieving and updating these preferences.

This separation also makes it possible to extend user preferences without significantly modifying the authentication or user-management subsystems.

4.16 Generated Image Implementation

NexusTeam contains a dedicated generated-image subsystem.

The backend includes:

GeneratedImagesController;

GeneratedImageService;

IGeneratedImageRepository;

generated-image DTOs and models.

4.17 WebSocket Implementation

The realtime communication system is implemented through WebSockets.

The main server component is WebSocketHandler.

The server exposes a WebSocket endpoint and handles the connection lifecycle.

The implementation performs several operations:

Accepting a WebSocket connection.

Authenticating the connection.

4.18 WebSocket Contracts

The NexusTeam.Shared project contains common WebSocket contracts.

Examples include:

ChatMessageContract;

TypingIndicatorContract;

MessageReadContract;

MessageDeliveredContract;

4.19 Realtime Typing Indicators

Typing indicators are implemented as realtime events.

When a user begins typing, the client can send a typing event through the WebSocket connection.

The event is processed by the server and distributed to relevant participants.

This allows the UI of another client to display an indication that another participant is currently typing.

Because typing information is temporary and realtime, it does not need to be persisted as a normal message.

4.20 Message Delivery and Read Status

The system includes separate realtime contracts for delivery and read states.

This allows the client to distinguish between different message lifecycle events.

For example:

Message sent

↓

Message delivered

4.21 Presence and Status

User presence is implemented through dedicated services including `UserStatusService` and `PresenceTrackingService`.

The status subsystem tracks changes in user availability and can communicate these changes to connected clients.

The implementation is supported by Redis and the realtime communication layer.

The conceptual flow is:

Connection State

|

4.22 WebSocket Connection Management

Active connections are managed through `WebSocketConnectionManager`.

The manager maintains the relationship between users and their active `WebSocket` connections.

This allows application services to send an event to a specific user without needing to know the implementation details of the underlying connection.

For example, after a new message is created, the server can identify the participants who should receive the message and use the connection manager to deliver the corresponding realtime event.

4.23 WebSocket Authentication

`WebSocket` connections are not treated as automatically trusted.

After a connection is established, the server expects authentication information before allowing protected operations.

The handler includes a limited authentication period. If authentication is not completed within the allowed time, the connection can be terminated.

This prevents unauthenticated connections from remaining active indefinitely.

4.24 Call Implementation

NexusTeam includes a voice-message subsystem implemented through dedicated client and shared components.

The server supports voice-message communication contracts such as:

call answer;

call termination;

4.25 Client Project Structure

The web client is contained in the `NexusTeam.Client` project.

The main areas are:

Views;

ViewModels;

Services;

Infrastructure;

4.26 web client Views

The client contains views for the main application workflows.

Examples include:

WelcomeView;

LoginView;
RegisterView;
ChatView;

4.27 ViewModels

ViewModels expose the data and commands required by the UI.

Important ViewModels include:

MainWindowViewModel;
ChatViewModel;
ConversationViewModel;
MessageViewModel;

4.28 MessagingService

MessagingService is one of the most important services on the client side.

It is responsible for coordinating communication with the backend.

Its responsibilities include operations related to:

retrieving messages;
sending messages;
editing messages;

4.29 AuthenticationService

The client contains a dedicated AuthenticationService.

It manages client-side authentication state and communicates with the backend authentication endpoints.

The authentication workflow is separated from the UI.

Therefore, LoginViewModel does not need to know how tokens are created or how HTTP requests are implemented. It invokes the authentication service and reacts to the result.

This separation improves maintainability and allows the authentication implementation to be changed independently from the login interface.

4.30 Credential Storage

The client contains a CredentialStorageService.

LiteDB is used for local client storage.

The purpose of the local storage mechanism is to maintain required client-side information without placing this responsibility directly inside the UI layer.

The credential storage implementation is separated from authentication itself, allowing the application to distinguish between:

authentication with the server;
storage of client-side authentication information.

4.31 Offline Message Queue

The web client contains an OfflineMessageQueue.

This component is designed to temporarily retain messages that cannot immediately be transmitted because the network connection is unavailable.

The general workflow is:

User sends message



4.32 Reconnection Strategy

The client also contains reconnection functionality for WebSocket communication.

If the connection is interrupted, the client can attempt to establish a new connection.

This is important because realtime messaging depends on a persistent connection.

The reconnection process must also consider authentication and the current client state.

A successful reconnection allows realtime communication to resume without requiring the user to restart the application.

4.33 File Attachment Client Implementation

The client contains a dedicated FileAttachmentService.

The service separates file-related operations from the rest of the messaging logic.

The UI can request an attachment operation without directly implementing the upload protocol.

This architecture is useful because file handling can involve:

file selection;

validation;

4.34 Avatar Implementation

User avatars are handled by a dedicated AvatarService.

The service provides an abstraction for loading and managing avatar data.

Separating avatar functionality from general user management keeps image-related operations out of unrelated components.

This also allows avatar handling to be changed without redesigning the complete user-management system.

4.35 Translation Implementation

The client contains a dedicated TranslationService.

Translation is therefore implemented as a separate client feature instead of being integrated directly into the main message ViewModel. The implementation calls the DeepL API.

This makes it possible to extend or replace translation functionality without significantly modifying the rest of the messaging subsystem.

4.36 Image Generation Client

The web client provides an image-generation workflow.

The main components include:

GeneratorView;

GeneratorViewModel;

ImageGeneratorService;

image-related models and views.

4.37 Navigation

The web client contains a NavigationService.

The purpose of this service is to centralize navigation between application views.

Without a dedicated navigation service, individual ViewModels would need to contain knowledge about the structure of the entire UI.

Centralized navigation reduces this coupling.

The general flow is:

User Action

4.38 Error Handling

Error handling is implemented at multiple levels.

The backend contains centralized exception handling middleware.

This provides a common mechanism for converting unexpected server-side exceptions into appropriate API responses.

The client contains an ErrorHandlingService for processing errors received during communication.

The layered approach can be represented as:

Database / Service Error

4.39 Logging

The project uses Serilog for application logging.

Logging is useful for:

debugging;

diagnosing connection problems;

monitoring application behavior;

identifying errors;

4.40 Database Seeding

The project contains a separate NexusTeam.DbSeeder application.

Its purpose is to initialize the required database infrastructure.

The seeding subsystem includes components for:

Oracle;

MongoDB;

Redis;

4.41 Implementation of Shared Models and DTOs

The NexusTeam.Shared project contains common models and DTOs used by different components.

DTOs provide explicit structures for communication between the client and backend.

Examples include request types for:

login;

registration;

sending messages;

4.42 Client-Server Communication

The client communicates with the backend using two main mechanisms.

REST

REST is used for operations such as:

authentication;

loading data;

modifying resources;

4.43 Web Client Implementation

In addition to the web application, NexusTeam contains a browser-based client.

The web client is implemented using standard web technologies:

HTML;

CSS;

JavaScript.

The main application logic is contained in app.js.

4.44 Web Client Deployment

The web client is packaged using Docker and served through Nginx.

The web deployment therefore consists of:

Browser

|



Nginx

4.45 Implementation Quality and Maintainability

Several implementation decisions contribute to maintainability.

Interface-Based Design

Interfaces are used for major services and repositories.

Dependency Injection

Dependencies are supplied through the DI container rather than created directly.

Shared Contracts

4.46 Implementation Challenges

The implementation of NexusTeam involves several technically complex areas.

The first major challenge is realtime communication. Unlike ordinary HTTP requests, WebSockets require connection lifecycle management, authentication, reconnection, message routing, and state synchronization.

The second challenge is integration between different storage technologies. The application must maintain consistent behavior while accessing Oracle, MongoDB, and Redis.

Another challenge is supporting multiple clients. Changes to the backend API or shared contracts can affect both the web client and web applications.

Finally, the project contains a significant amount of client-side functionality. Maintaining separation between Views, ViewModels, services, and communication components is therefore essential.

4.47 Implementation Limitations

Although the architecture provides a strong foundation, several implementation areas can be improved further.

Automated test coverage should be expanded to cover more services, repositories, and client components.

Some advanced communication functionality, particularly advanced communication functionality, requires additional work before it can be considered equivalent to a fully production-grade WebRTC implementation.

Production deployment would also require stronger secret management than development-oriented configuration files.

Additional observability, performance monitoring, and automated integration testing would further improve the implementation.

These limitations do not prevent the system from demonstrating the intended software engineering concepts, but they represent important areas for future development.

4.48 Summary

The implementation of NexusTeam follows the layered and modular architecture described in Chapter 3.

The ASP.NET Core backend contains dedicated controllers, services, repositories, middleware, authentication components, and infrastructure services. Oracle, MongoDB, and Redis are used according to the type of data and performance requirements.

Realtime communication is implemented through WebSockets and a set of shared contracts. This provides support for messaging, typing indicators, presence, message status, and voice-message communication.

The browser-based web application follows modular web UI architecture and uses dedicated services for authentication, messaging, calls, attachments, avatars, translation, image generation, navigation, and offline communication.

A browser-based client provides an additional access method and uses the same backend infrastructure.

The separation of responsibilities, dependency injection, shared contracts, repository abstraction, and dedicated services make the implementation modular and extensible.

5. User Interface

5.1 Introduction

The user interface is one of the most important parts of NexusTeam because it represents the main point of interaction between users and the communication platform. The interface was designed to provide access to messaging, user management, file sharing, chat organization, calls, settings, and additional functionality while keeping the main communication workflow understandable and efficient.

NexusTeam provides two client interfaces:

- a native browser-based web application based on web client;

- a browser-based web application implemented using HTML, CSS, and JavaScript.

Although the technologies and visual implementations of the two clients differ, they are connected to the same backend and provide access to the same central communication platform.

The interface design is based on several principles:

5.2 UI Design Goals

The main objective of the NexusTeam interface is to make communication the central element of the application.

A messaging application is used frequently and for relatively short interaction cycles. Therefore, users should be able to perform common actions without navigating through unnecessary screens.

The main UI goals are:

- Simplicity

The interface should avoid unnecessary complexity in common workflows such as opening a chat, sending a message, or switching between conversations.

- Consistency

5.3 User Interface Structure

The main web application can be conceptually divided into several areas:

- Application Header

- Navigation
- Conversation
- Additional

5.4 Welcome and Authentication Interface

The first interaction with the application is provided by the welcome and authentication interface.

The authentication workflow includes:

- welcome screen;

- login;

- registration;

- authentication feedback.

5.5 Login Interface

The login screen is designed around a simple objective: authenticate the user with the minimum number of required interactions.

The main components are:

- username or email input;

- password input;
- login action;
- navigation to registration;

5.6 Registration Interface

The registration interface allows new users to create their account.

The form contains the necessary account information and provides validation feedback if the submitted information does not satisfy the application's requirements.

The registration workflow follows the same visual principles as login:

- enter information;
- validate the information;
- submit the request;

5.7 Main Application Window

After authentication, users are taken to the main application environment.

The main window acts as the central navigation point of the web client.

It provides access to:

- conversations;
- chat folders;
- user information;

5.8 Chat List

The chat list provides an overview of the user's conversations.

A typical chat entry can contain information such as:

- participant or group name;
- avatar;
- latest message information;
- conversation state;

5.9 Chat Folders

As the number of conversations increases, a simple chronological chat list can become difficult to navigate.

NexusTeam therefore provides chat folders.

Folders allow users to organize conversations into categories according to their own preferences.

The interface includes functionality for creating and managing folders.

This feature is particularly useful for users who participate in many conversations and need a mechanism to separate different groups of chats.

The folder functionality also demonstrates the principle of separating content organization from the actual conversation data.

5.10 Conversation View

The conversation view is the central interface of NexusTeam.

It displays the messages belonging to the currently selected chat and provides the controls required for sending new content.

The main elements include:

- conversation header;

- participant information;
- message history;

5.11 Message Presentation

Messages are displayed as individual conversation elements.

The message presentation distinguishes between messages sent by the current user and messages received from other participants.

This visual distinction helps users understand the direction of communication without requiring them to inspect the sender name for every message.

Message elements can contain:

- text;
- timestamps;

5.12 Message Actions

The interface provides actions associated with individual messages.

Depending on the message and the user's permissions, available actions can include:

- editing;
- deleting;
- viewing attachments;
- copying or interacting with content.

5.13 Message Input

The message input area is one of the most frequently used controls in the application.

It provides a simple mechanism for entering and sending text.

The input area is also integrated with additional communication features such as attachments and other message-related actions.

The design goal is to keep the primary action — sending a message — immediately accessible while still allowing users to access advanced features when required.

5.14 Typing Indicators

The interface can display typing information when another participant is composing a message.

Typing indicators provide realtime feedback without requiring the user to refresh the conversation.

From a UX perspective, this makes the application feel more responsive and similar to modern messaging platforms.

The information is intentionally temporary. It is therefore presented as part of the current conversation state rather than as a permanent message.

5.15 Message Delivery and Read State

The UI can represent message lifecycle information through message status.

The communication model distinguishes between events such as:

- delivered;
- read.

This provides users with additional information about whether their messages have reached the recipient and whether they have been viewed.

The interface should represent these states unobtrusively because they provide useful context without being the primary content of a conversation.

5.16 User Presence

User presence information provides additional context during communication.

The interface can reflect changes in the status of users through realtime updates.

For example, the application can distinguish between users who are currently available and users whose status has changed.

Presence information is especially useful in direct communication because it gives the sender an indication of whether the recipient may currently be available.

5.17 User Profile

NexusTeam provides functionality for viewing and managing user profile information.

The profile-related interface includes information such as:

username;

avatar;

profile information;

status.

5.18 Settings Interface

Settings are provided through a dedicated view.

Separating settings from the main conversation interface prevents configuration controls from interfering with everyday communication.

The settings area can be used for user preferences and other configurable aspects of the application.

This separation follows the principle that frequently used communication functions should remain immediately accessible while less frequently modified options should be placed in a dedicated configuration area.

5.19 File Attachments

The attachment interface allows users to add files to messages.

The workflow is designed around the following sequence:

Select attachment

↓

Review / process file

↓

5.20 Images

Images require different presentation from ordinary text files.

The application therefore provides image-related functionality, including image viewing and a dedicated image viewer interface.

The ImageViewerDialog provides a focused environment for inspecting images without replacing the complete conversation context.

This approach improves usability because users can inspect visual content in greater detail while maintaining a clear relationship with the original message.

5.21 Files List

The application also provides a dedicated files-related interface.

The `FilesListView` allows file content to be presented separately from the conversation itself.

This is useful when users need to locate shared files without manually scrolling through a long conversation.

The feature demonstrates two different approaches to information presentation: chronological presentation inside the conversation; content-oriented presentation through the files interface.

5.22 Emoji Picker

NexusTeam contains an emoji picker component.

The purpose of the picker is to make expressive content accessible without requiring users to remember or manually enter Unicode characters.

The picker is integrated into the message composition workflow so that adding an emoji does not require leaving the current conversation.

This is another example of keeping secondary functionality close to the primary communication action.

5.23 Voice Messages

Voice-related functionality is integrated into the communication interface.

The client contains a dedicated `VoiceMessageService`, which separates voice-message processing from the general messaging interface.

The UI therefore provides a mechanism for users to interact with voice content without requiring the messaging `ViewModel` to implement the underlying audio processing itself.

5.24 Voice Messages

The call functionality is represented through a dedicated call interface.

The web client contains:

`VoiceMessageRecorder`;

`VoiceMessageViewModel`;

`VoiceMessageService`.

The interface provides controls for the call lifecycle, including the ability to respond to call requests and terminate calls.

5.25 Image Generation Interface

NexusTeam provides a dedicated image-generation interface.

The relevant components include:

`GeneratorView`;

`GeneratorViewModel`;

`ImageGeneratorService`.

The interface allows the user to enter the required information for generating an image and then presents the resulting content.

5.26 Translation Interface

Translation functionality is exposed through a dedicated translation interface.

The project contains a TranslateWindow and corresponding translation service.

This allows users to access translation functionality without embedding translation logic directly into every message component.

The interface can therefore act as an additional communication aid, particularly in multilingual conversations.

5.27 Dialogs

The web client contains several dedicated dialogs.

Examples include:

create chat;

create folder;

edit group;

image viewer;

5.28 Code Preview

The project also includes a code preview interface.

The presence of CodePreviewWindow and CodePreviewControl indicates that the client can present code-related content using a specialized interface.

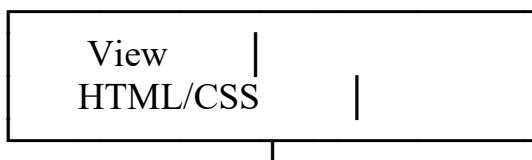
This is useful for a communication platform intended to support different types of content rather than only plain text.

A specialized presentation mechanism improves readability compared with displaying source code as ordinary paragraph text.

5.29 Desktop UI Architecture

The desktop UI follows the modular web UI architecture structure described in the architecture and implementation chapters.

The relationship can be summarized as:



5.30 Data Binding

web client data binding is used to connect UI elements with ViewModel properties.

This allows changes in application state to be reflected in the interface without requiring manual manipulation of every visual element.

For example, when a conversation changes, the ViewModel can update its collection of messages and the corresponding UI can update automatically.

The same mechanism can be used for:

user status;

selected chat;

5.31 Commands and User Actions

User actions are connected to ViewModel commands rather than directly invoking application services from the UI.

Typical actions include:

- login;
- registration;
- selecting a chat;
- sending a message;

5.32 Web User Interface

In addition to the browser-based web application, NexusTeam provides a browser-based interface.

The web client is implemented using:

- HTML;
- CSS;
- JavaScript.

The web application provides access to core messenger functionality without requiring a native Windows installation.

5.33 Web UI Structure

The web client follows the same general communication model as the web client.

The browser communicates with the backend using REST and WebSocket connections.

5.34 Responsive and Adaptive Considerations

The web interface has different requirements from the web client desktop interface because browser users may access the application using different window sizes.

The UI therefore uses web layout mechanisms to organize content within the available browser space.

The web application, in contrast, can take advantage of the predictable Windows environment and web client's layout system.

Although the implementation technologies differ, both interfaces aim to maintain the same conceptual structure:

- navigation;
- conversations;

5.35 UI Consistency Across Platforms

Supporting multiple clients introduces a potential problem: users may encounter different interaction models depending on the platform.

To reduce this problem, the two clients share the same underlying concepts.

For example, both clients provide access to:

- users;
- chats;
- messages;

5.36 Feedback and Error States

A communication application must communicate system state to the user.

Examples include:

- invalid login credentials;
- invalid registration data;
- failed message delivery;
- connection problems;

5.37 Loading States

Network operations can take time and should not make the interface appear frozen.

Loading states can be used for operations such as:

- authentication;
- loading conversations;
- retrieving message history;
- uploading attachments;

5.38 Usability Considerations

Several usability principles are reflected in the interface.

Recognition Instead of Recall

Frequently used functions are represented by visible controls rather than requiring users to remember commands.

Consistent Placement

Related controls are grouped together so that users can develop predictable interaction patterns.

Immediate Feedback

5.39 Accessibility Considerations

Accessibility is an important consideration for any user-facing software.

The interface can be improved through:

- sufficient contrast;
- readable text;
- recognizable controls;
- keyboard navigation;

5.40 UI Security Considerations

The interface itself should not expose sensitive information unnecessarily.

Authentication credentials should be handled through protected input fields.

The UI should not display:

- passwords;
- access tokens;
- refresh tokens;

5.41 UI and Backend Integration

The interface is closely connected to the backend but does not directly access its internal components.

5.42 UI Strengths

The main strengths of the NexusTeam UI are:

Feature Coverage

The interface provides access to a broad range of messenger functionality.

Modular Structure

Different functions are represented through dedicated views, dialogs, ViewModels, and services.

Multiple Platforms

5.43 UI Limitations and Future Improvements

The current interface provides a broad range of functionality, but several improvements could be considered for future versions.

First, the UI could be further standardized between the desktop and web clients. A shared design system could define common colors, typography, spacing, icons, and interaction patterns.

Second, accessibility testing could be expanded using automated accessibility tools and dedicated user testing.

Third, more responsive layouts could be introduced for different screen sizes, particularly in the web client.

Fourth, additional usability testing could be used to determine whether less frequently used functionality is placed in the most appropriate locations.

Finally, advanced features such as calls and generated content could benefit from more detailed progress, error, and state indicators.

5.44 Summary

The NexusTeam user interface provides access to the system's communication and supporting functionality through two client applications.

The browser-based web application follows the modular web UI architecture pattern and contains dedicated views, ViewModels, services, dialogs, and controls. Its primary focus is the conversation interface, supported by functionality for attachments, folders, profiles, settings, calls, translation, image generation, and other communication features.

The web client provides a browser-based alternative implemented using HTML, CSS, and JavaScript. It communicates with the same backend through REST and WebSocket connections.

The interface is designed around simplicity, consistency, realtime feedback, and accessibility of frequently used functionality. The separation between the UI, ViewModels, services, and backend allows the presentation layer to evolve independently from the underlying implementation.

Overall, the UI serves as the main connection between the technical functionality of NexusTeam and its users, translating backend operations into understandable and interactive workflows.

Figure 5.1 — Use Case Diagram

6. Testing

6.1 Introduction

Testing is an essential part of the NexusTeam development process because the system consists of multiple interconnected components, including a backend server, two client applications, several databases, realtime communication, authentication mechanisms, and containerized infrastructure.

A failure in one component can affect other parts of the system. For example, a change to a shared WebSocket contract can affect both the server and the clients, while a database configuration problem can prevent authentication or messaging functionality from working correctly.

The testing approach therefore considers the system at several levels:

functional testing;

integration testing;

API testing;

database testing;

6.2 Testing Objectives

The main objectives of testing NexusTeam are:

To verify that the implemented functionality satisfies the defined requirements.

To identify defects before deployment.

To verify communication between clients and the backend.

To verify correct database operations.

To verify authentication and authorization mechanisms.

To verify realtime WebSocket communication.

6.3 Testing Strategy

The testing strategy follows a layered approach.

System Testing

Client Backend Infrastructure

6.4 Test Environment

The NexusTeam test environment consists of the same major components as the development environment.

The main components are:

NexusTeam Server;

NexusTeam Client;

Web Client;

Oracle Database;

MongoDB;

6.5 Test Data

Testing should use dedicated test accounts and test conversations rather than real user data.

Typical test data includes:

valid user accounts;

invalid registration data;

- incorrect passwords;
- direct conversations;
- group conversations;

6.6 Functional Testing

Functional testing verifies whether the system performs the operations defined in the functional requirements.

The main functional areas are:

- registration;
- authentication;
- user management;
- chat creation;
- messaging;

6.7 Authentication Testing

Authentication is one of the highest-priority areas for testing.

The following scenarios should be verified:

Valid Registration

Input: valid registration information.

Expected result: a new account is created successfully.

Duplicate Registration

Input: registration using an already existing username or email.

6.8 Authentication Test Cases

These tests verify both normal authentication behavior and common invalid scenarios.

6.9 User Management Testing

User management functionality should be tested using both valid and invalid inputs.

Important scenarios include:

- retrieving an existing user;
- searching for users;
- updating profile information;
- changing an avatar;
- accessing another user's information;

6.10 Chat Testing

Chat functionality is tested by creating and manipulating conversations.

Important scenarios include:

Create a direct conversation.

Create a group conversation.

Retrieve an existing conversation.

Add or manage participants where supported.

Access a conversation as an authorized participant.

6.11 Message Testing

Messaging represents the central functional area of the application.

Tests should verify:
sending messages;
retrieving message history;
editing messages;
deleting messages;
message ordering;

6.12 Message Test Cases

6.13 Realtime WebSocket Testing

WebSocket testing is especially important because realtime communication is a central feature of NexusTeam.

The following aspects should be tested:
connection establishment;
authentication;
authentication timeout;
message reception;
message broadcasting;

6.14 WebSocket Test Cases

6.15 Typing Indicator Testing

Typing indicators are temporary realtime events and should not be stored as normal messages.

A typical test involves two clients.

Client A starts typing.

Expected result: Client B receives the corresponding typing event.

When Client A stops typing:

Expected result: Client B receives the event indicating that typing has stopped.

The test should also verify that a typing event is not incorrectly displayed after the sender has stopped typing.

6.16 Presence Testing

Presence functionality should be tested by changing the connection state or user status of a test account.

For example:

User A logs in.

User B observes User A's status.

User A changes status or disconnects.

User B receives the corresponding update.

The expected result is that the interface reflects the latest known state.

6.17 Attachment Testing

Attachment functionality requires both functional and validation testing.

Important scenarios include:

selecting a valid file;

uploading a file;

retrieving an attachment;

- attaching a file to a message;
- handling an invalid file;

6.18 Chat Folder Testing

Chat folders are tested separately from the underlying conversations.

Tests include:

- creating a folder;
- renaming a folder;
- retrieving folders;
- adding conversations;
- removing conversations;

6.19 Voice Message Testing

Voice messages require testing of recording, cancellation, upload, storage, delivery, playback, and invalid-input handling.

The main scenarios include:

User A starts a voice-message recording.

User A stops and sends the recording.

User B receives the voice message.

User B can play the received recording.

A recording can be cancelled before sending.

The system handles microphone or permission errors without exposing technical details.

Additional negative cases include an unavailable microphone, interrupted upload, unsupported audio data, and network disconnection during upload.

6.20 Database Testing

NexusTeam uses three different infrastructure technologies for data-related operations.

Oracle

Oracle-related testing focuses primarily on:

- user creation;
- user lookup;
- uniqueness constraints;
- account updates;

6.21 Database Consistency Testing

Database consistency is particularly important because the project uses multiple persistence technologies.

A functional workflow may involve more than one data store.

For example, a user session may involve authentication information and Redis session state, while a message workflow may involve MongoDB persistence and realtime state.

Tests should verify that the system does not expose inconsistent states to the user.

For example:

Message sent

↓

6.22 API Testing

REST endpoints should be tested independently from the UI.

API testing allows developers to determine whether a problem originates in the backend or in the client.

The API testing process includes:

- sending valid requests;
- sending invalid requests;
- testing authentication;
- testing authorization;

6.23 API Validation Testing

Validation should be tested with boundary and invalid inputs.

Examples include:

- missing required fields;
- invalid identifiers;
- excessively long strings;
- invalid email formats;
- invalid credentials;

6.24 Security Testing

Security testing focuses on preventing unauthorized access and protecting sensitive information.

Important security scenarios include:

Authentication Bypass

Attempt to access protected endpoints without authentication.

Expected result: access denied.

Invalid Token

Attempt to access protected functionality using an invalid token.

6.25 Security Test Cases

6.26 Client Testing

The web client is tested from the user's perspective as well as through its individual services.

Important areas include:

- startup;
- authentication;
- navigation;
- chat selection;
- message sending;

6.27 web client UI Testing

UI testing focuses on user workflows.

Example workflow:

Start Application

↓

Login



Main Window

6.28 Web Client Testing

The web client should be tested independently because browser behavior differs from the web client environment.

Testing should include:

registration;

login;

chat loading;

sending messages;

realtime events;

6.29 Cross-Client Testing

One of the most important system-level tests is communication between different clients.

For example:

web client Client



Send message

Server

6.30 Reconnection Testing

Network interruptions are simulated by temporarily disabling or interrupting the client connection.

The expected behavior is:

The client detects the connection failure.

The realtime connection is closed or marked unavailable.

The client attempts reconnection.

The connection is restored.

Realtime communication resumes.

6.31 Offline Message Testing

The web client contains an offline message queue.

The test procedure is:

Establish a normal connection.

Disconnect the client from the network.

Create a message.

Verify that the message is retained locally.

Restore network connectivity.

6.32 Performance Testing

Performance testing evaluates whether the system remains responsive under expected workloads.

Potential performance tests include:

sending multiple messages;

- loading a large message history;
- establishing multiple WebSocket connections;
- uploading attachments;
- performing repeated API requests;

6.33 Load Testing

A production messaging platform may need to support many simultaneous users.

Load testing can simulate:

- multiple concurrent users;
- multiple active chats;
- simultaneous WebSocket connections;
- high message throughput;
- repeated authentication requests.

6.34 Error Handling Testing

The project contains centralized server exception handling and client-side error handling.

Testing should verify that errors are converted into appropriate responses.

Examples include:

- invalid requests;
- unavailable resources;
- authentication failures;
- database failures;

6.35 Logging Testing

Logging should be verified by intentionally triggering controlled events such as:

- authentication failures;
- invalid API requests;
- connection failures;
- unexpected exceptions.

The expected result is that relevant diagnostic information is recorded.

At the same time, sensitive information should not be written into logs.

6.36 Deployment Testing

The Docker-based environment should also be tested.

The deployment test includes:

- Starting the Docker environment.
- Starting databases and infrastructure.
- Initializing database structures.
- Starting the backend.
- Starting the web client.

6.37 End-to-End Testing

End-to-end testing verifies complete user workflows.

A representative scenario is:

E2E-01 — Complete Messaging Workflow

Start the system.

Register User A.
Register User B.
Login as User A.

6.38 Example End-to-End Test Matrix

6.39 Defect Classification

Discovered problems can be categorized according to their severity.

Critical

The system cannot start or a fundamental security or data-integrity problem exists.

High

A major core feature such as authentication or messaging does not work.

Medium

A secondary feature does not work correctly, but the core communication workflow remains available.

6.40 Defect Management

When a defect is identified, it should be documented with:

defect description;

reproduction steps;

expected behavior;

actual behavior;

severity;

affected component;

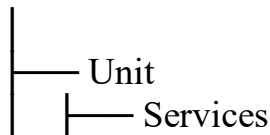
6.41 Automated Testing

The current project contains extensive application code but does not include a separate comprehensive automated test project covering the backend and clients.

This represents one of the main testing limitations.

A future automated testing structure could contain:

NexusTeam.Tests



6.42 Unit Testing Recommendations

The highest-value candidates for unit testing are components with deterministic business logic.

Examples include:

validators;

authentication services;

token services;

message services;

chat services;

6.43 Integration Testing Recommendations

Integration tests should verify interactions between components.

Recommended integration scenarios include:

- authentication with Oracle;
- message persistence with MongoDB;
- session operations with Redis;
- REST controllers and services;
- WebSocket connection establishment;

6.44 Testing Limitations

Several limitations should be considered when evaluating the current testing approach.

First, the project does not currently contain a comprehensive automated test suite.

Second, large-scale performance and load testing have not been fully implemented.

Third, complete end-to-end testing of every feature across both clients would require a significantly larger test environment.

Fourth, some advanced functionality, particularly realtime calls, requires additional testing under different network conditions.

Finally, UI testing has limitations because visual and usability behavior is difficult to verify exclusively through backend tests.

These limitations should be considered when evaluating the maturity of the current system.

6.45 Future Testing Improvements

Future versions of NexusTeam could improve testing through:

Automated Unit Tests

Create unit tests for services, validators, and business logic.

Integration Tests

Automate interactions with Oracle, MongoDB, and Redis.

WebSocket Test Suite

Create automated tests for connection lifecycle and realtime events.

6.46 Testing and Requirements Traceability

Testing can be directly connected to the requirements defined in Chapter 2.

This traceability ensures that testing remains connected to the original project requirements.

6.47 Overall Test Evaluation

The testing strategy covers the major components and workflows of the NexusTeam system.

The highest priority is given to authentication, messaging, realtime communication, data persistence, and security because these features form the core of the application.

Testing the system at multiple levels is particularly important because NexusTeam contains several independent technologies. A successful API test alone does not guarantee that a message is correctly displayed in the web client or web client. Similarly, a successful database operation does not guarantee that the corresponding WebSocket event is delivered to another user.

For this reason, integration and end-to-end testing are essential complements to individual component testing.

6.48 Summary

Testing NexusTeam requires a multi-level approach because the system consists of several interconnected applications and infrastructure components.

Functional testing verifies authentication, users, chats, messages, attachments, folders, presence, calls, and additional features. API testing verifies the REST interface, while WebSocket testing verifies realtime communication.

Database testing covers Oracle, MongoDB, and Redis. Client and UI testing verify the web client and web applications from the user's perspective. Security testing focuses on authentication, authorization, token handling, rate limiting, and protected WebSocket connections.

End-to-end testing connects these individual areas and verifies complete user workflows.

The current project provides a strong basis for manual and integration-oriented testing, but a comprehensive automated test suite should be considered a major future improvement. Automated unit, integration, WebSocket, UI, security, and load tests would significantly increase confidence in the system and make future development safer.

The testing results and future improvements described in this chapter provide a foundation for evaluating whether the implementation satisfies the requirements defined earlier in the documentation.

7. Project Management

7.1 Introduction

Project management played an important role in the development of NexusTeam because the system consists of several interconnected components that were developed by different team members.

The project required coordination between backend development, client implementation, user interface design, testing, requirements management, architecture, and documentation. Changes in one area could directly affect another area. For example, changes to backend API contracts could require corresponding changes in the client, while modifications to shared WebSocket contracts could affect both the server and multiple clients.

The project management approach was therefore based on clear responsibility distribution, version control, milestone-based development, regular coordination, and integration of the individual components into a common system.

7.2 Team Organization

The development team was divided according to the major areas of the project.

Although each team member had a primary responsibility, the project was developed collaboratively.

This was particularly important because the components were not completely independent.

7.3 Responsibility Distribution

The responsibility distribution was designed to reduce overlap while maintaining communication between the different work areas.

Requirements Management

The requirements-management responsibility included:

7.4 Development Workflow

The development process followed an iterative workflow rather than implementing the complete system in a single sequence.

A simplified development cycle was:

Requirements

7.5 Milestone-Based Development

The project was organized around development milestones.

Milestones provided intermediate targets that made it possible to track progress and divide the project into manageable stages.

The general progression was:

7.6 Planning and Task Breakdown

Large features were divided into smaller technical tasks.

For example, the messaging functionality could be broken down into:

Messaging

7.7 Task Dependencies

Some tasks could be performed independently, while others depended on previously completed work.

For example, the UI for a new backend feature could be designed independently, but its final integration depended on the API being available.

A simplified dependency chain is:

7.8 Version Control

Git was used as the primary version-control system.

The project repository provides a central location for:
source code;

7.9 Branching Strategy

Development work was separated into branches where appropriate.

A typical workflow was:
main

7.10 Commits

Commits were used to represent logical units of work.

A good commit should describe a meaningful change rather than a collection of unrelated modifications.

Examples of appropriate commit concepts include:

7.11 Integration Process

Integration was one of the most important project-management activities because the project contains multiple interconnected components.

A feature was considered integrated when the relevant components could operate together.

For example, a messaging feature required integration of:

7.12 Shared Contracts and Team Coordination

The NexusTeam.Shared project was particularly important for team coordination.

The project contains common models, DTOs, enums, and WebSocket contracts.

Without shared definitions, the server and clients would have to maintain independent copies of the same structures.

7.13 Communication Between Team Members

Regular communication was required because individual tasks were technically dependent on each other.

The most important communication topics included:

API changes;

7.14 Change Management

Requirements and implementation details can change during software development.

Changes were evaluated according to their potential impact on other components.

A typical change-management process was:

7.15 Issue and Defect Management

Problems discovered during development or testing were treated as tasks that needed to be investigated and resolved.

A defect workflow can be represented as:

Problem

7.16 Risk Management

Risk management was used to identify technical and organizational problems that could negatively affect the project.

The main risks included:

Integration Risk

7.17 Technical Debt Management

During development, some implementation decisions may be acceptable for an academic prototype but require improvement in a production environment.

Examples include:

incomplete automated test coverage;

7.18 Project Quality Control

Quality control was performed through several mechanisms.

Code Review

Changes to important shared components should be reviewed before integration.

7.19 Definition of Done

A feature can be considered complete when the relevant development activities have been performed.

A practical Definition of Done for NexusTeam is:

requirement is understood;

7.20 Documentation as a Project Activity

Documentation was not treated solely as a final administrative task.

The project documentation covers:

requirements;

7.21 Project Management and Architecture

Project management influenced the architecture of NexusTeam.

Because different team members worked on different areas, clear component boundaries were necessary.

For example:

7.22 Project Management and Testing

Testing was integrated into the development workflow rather than being considered only after all implementation work had finished.

When a feature was implemented, the corresponding functionality could be tested independently.

For example:

7.23 Project Management and Requirements

Requirements served as the basis for task prioritization.

Core requirements such as authentication, messaging, realtime communication, and persistence were prioritized because they were necessary for the basic system to function.

Additional features such as image generation, translation, and advanced communication functionality could be developed after the core platform was established.

7.24 Project Progress Tracking

Project progress was evaluated through the completion of major components and milestones.

Progress indicators included:
requirements completed;

7.25 Collaboration Between Specializations

The specialization of responsibilities created several important collaboration points.

Requirements ↔ Architecture

Requirements determined which architectural capabilities were necessary.

7.26 Project Management Challenges

Several challenges were encountered as a result of the system's complexity.

The first challenge was coordinating changes between multiple components. A seemingly small backend change could require modifications in both clients.

The second challenge was integrating several infrastructure technologies. Oracle, MongoDB, Redis, and Docker each introduce their own configuration and operational requirements.

7.27 Lessons Learned

The project provided several lessons regarding collaborative software development.

Clear Interfaces Are Essential

Shared APIs and contracts significantly reduce misunderstandings between developers.

7.28 Future Project Management Improvements

Future development could improve the project-management process through additional automation and formalization.

Possible improvements include:

Automated CI Pipeline

7.29 Project Management Summary

The development of NexusTeam required coordination between several technical areas and team members.

Responsibilities were divided into requirements and architecture, backend and client implementation, UI development, and testing. Documentation was maintained as a shared project activity.

Git was used for version control, while branches, commits, milestones, and integration processes supported collaborative development.

7.30 Final Project Management Perspective

NexusTeam demonstrates that successful software development depends not only on implementing individual features but also on coordinating the relationships between them.

The project combined requirements management, architecture, implementation, UI development, testing, and documentation into one development process.

The division of responsibilities allowed team members to focus on specialized areas while the shared architecture, common contracts, version control, and integration activities maintained consistency across the project.

8. Documentation

8.1 Introduction

Documentation is an important part of the NexusTeam project because the system consists of multiple applications, databases, communication protocols, and infrastructure components.

The purpose of the project documentation is not only to describe the final implementation but also to make the system understandable to developers, testers, administrators, and other stakeholders.

A developer joining the project should be able to understand the overall architecture, install the required infrastructure, start the applications, identify the main components, and continue development without having to reconstruct the entire system from the source code.

The documentation therefore covers several levels:
project overview;

8.2 Documentation Objectives

The main objectives of the documentation are:

To explain the purpose and scope of NexusTeam.

To describe the functional and non-functional requirements.

To explain the system architecture.

To document important implementation decisions.

8.3 Documentation Structure

The project documentation is divided into several major sections.

This structure corresponds to the major aspects required for the project evaluation.

8.4 Documentation for Different Stakeholders

Different stakeholders require different levels of technical information.

End Users

Users are primarily interested in:

how to access the system;

how to log in;

8.5 Project Overview Documentation

The project overview provides a high-level explanation of NexusTeam.

It describes:

the purpose of the application;

target users;

main functionality;

8.6 Requirements Documentation

Requirements documentation defines what the system is expected to provide.

The requirements chapter contains:

functional requirements;

non-functional requirements;

stakeholders;

8.7 Architecture Documentation

Architecture documentation explains how the major components of NexusTeam interact.

It describes:

client-server architecture;

backend layers;

web client/modular web UI architecture architecture;

8.8 Implementation Documentation

Implementation documentation provides more technical detail than the architecture chapter.

It explains the structure and responsibilities of:

controllers;

services;

repositories;

8.9 API Documentation

The backend provides a REST API through ASP.NET Core.

API documentation should describe:

available endpoints;

HTTP methods;

required authentication;

8.10 WebSocket Documentation

REST documentation alone is insufficient because realtime communication is an essential part of NexusTeam.

The WebSocket documentation should describe:

connection endpoint;

authentication process;

message envelope;

8.11 Database Documentation

Because NexusTeam uses multiple persistence technologies, database documentation is particularly important.

The documentation distinguishes between:

Oracle

Used primarily for structured user-related data.

MongoDB

8.12 Database Seeder Documentation

The NexusTeam.DbSeeder project provides database initialization functionality.

Documentation for the seeder should explain:

when it should be executed;

which databases it initializes;

what data it creates;

8.13 Installation Documentation

Installation documentation is required so that a new developer can reproduce the development environment.

A typical installation process consists of:

Clone the project repository.

Install the required development tools.

Configure the required environment variables.

8.14 Development Environment

The project uses several technologies that need to be available during development.

The main requirements include:

.NET 8 SDK/runtime;

development environment supporting C#;

Docker;

8.15 Configuration Documentation

Configuration information is an important part of deployment documentation.

The backend uses configuration sections for components such as:

Oracle;

MongoDB;

Redis;

8.16 Docker Documentation

Docker is used to simplify infrastructure setup.

The Docker documentation should explain:

available containers;

container dependencies;

network configuration;

8.17 Deployment Documentation

Deployment documentation describes how the complete application is made available.

The main deployment components include:

backend server;

web client;

database infrastructure;

8.18 Troubleshooting Documentation

Troubleshooting documentation should provide solutions for common problems.

Examples include:

Backend Does Not Start

Possible causes include:

unavailable database;

8.19 Logging Documentation

Logging is used to help developers and administrators understand system behavior.

Documentation should explain:
where logs are generated;
which events are logged;
how log levels are configured;

8.20 Security Documentation

Security documentation describes the mechanisms used to protect the system.
The main mechanisms include:
password hashing;
JWT authentication;
refresh tokens;

8.21 Maintenance Documentation

Software maintenance requires documentation that remains useful after the initial development phase.

Maintenance documentation should help developers answer questions such as:
Where is a feature implemented?
Which database stores the related data?
Which API endpoint is responsible?

8.22 Adding a New Feature

The documentation should also describe the general process for adding a new feature.

A new feature can follow these steps:
Define Requirement
↓
Update Architecture if necessary

8.23 Documentation and Version Control

Documentation should be managed together with the source code whenever possible.

This ensures that changes to implementation and documentation can be associated with the same project history.

For example, if a new API endpoint is introduced, the corresponding documentation should be updated in the same development cycle.

This prevents the documentation from becoming outdated.

8.24 Documentation Quality

High-quality documentation should satisfy several criteria.

Accuracy

Information must correspond to the actual implementation.

Completeness

Important system components and workflows should be covered.

8.28 API and UI Documentation Relationship

API documentation and UI documentation describe different layers of the same workflow.

For example, sending a message can be documented at three levels:

User Level

The user enters text and selects the send action.

Client Level

8.29 Testing Documentation

Testing documentation records how the system was verified.

It includes:

testing objectives;

test environment;

test cases;

8.33 Documentation and Future Developers

A future developer should be able to use the documentation to:

Understand the project purpose.

Set up the development environment.

Start the infrastructure.

Run the server.

8.34 Documentation Limitations

The current documentation can still be improved in several areas.

First, API documentation could be expanded with complete examples for every endpoint.

Second, deployment documentation could provide more detailed environment-specific instructions.

Third, automated API documentation could be integrated more closely with the source code.

Fourth, a dedicated user manual could be created for non-technical users.

8.35 Future Documentation Improvements

Several improvements are recommended for future versions.

Complete API Reference

Document every endpoint with request and response examples.

Sequence Diagrams

Add diagrams for:

8.36 Documentation Review

Before the final release, documentation should be reviewed for:

outdated information;

incorrect component names;

missing features;

inconsistent terminology;

8.37 Documentation Checklist

The following checklist can be used before project submission:

This checklist provides a final verification that the major documentation requirements have been addressed.

9. Conclusion and Future Work

9.1 Conclusion

NexusTeam was developed as a multi-platform communication system combining real-time messaging, user management, file sharing, chat organization, voice communication, and additional services in one application.

The project demonstrates the complete software development process, beginning with requirements analysis and architectural planning and continuing through implementation, user interface development, testing, project management, and documentation.

The resulting system consists of several interconnected components. The ASP.NET Core 8 backend provides the central application logic and communication interface. The browser-based web client provides a native Windows user interface, while the web client provides browser-based access to the same backend functionality.

One of the main architectural characteristics of the project is the separation of responsibilities between the different system layers. Controllers provide the API interface, services implement business logic, repositories abstract database access, and middleware handles cross-cutting concerns such as authentication, logging, validation, and error handling.

The system also uses several persistence technologies according to the type of data being processed. Oracle is used for structured user-related information, MongoDB is used for communication-oriented document data, and Redis provides fast access to temporary and frequently changing information.

Realtime communication is implemented using WebSockets. This enables functionality such as realtime messaging, typing indicators, message delivery and read states, user presence, and voice-message communication. Shared DTOs and WebSocket contracts provide a common communication model between the backend and clients.

The web client follows the modular web UI architecture pattern, which separates the user interface from application state and communication logic. The web client provides an alternative browser-based interface while using the same backend infrastructure.

From a requirements perspective, the project addresses the main requirements of a modern communication platform. Users can authenticate, manage conversations, exchange messages, share files, organize chats, and use additional communication features.

The testing process considered multiple levels, including functional testing, API testing, database testing, WebSocket testing, security testing, client testing, integration testing, and end-to-end workflows. This approach is particularly important for NexusTeam because the correctness of the complete system depends on the interaction between multiple components.

9.2 Achievement of Project Objectives

The main project objectives can be summarized as follows:

The project therefore achieves the main objectives established during the requirements and planning phases.

9.3 Technical Achievements

Several technical decisions represent important achievements of the project.

Modular Backend

The backend is divided into controllers, services, repositories, middleware, and infrastructure components. This makes individual parts easier to maintain and extend.

Multiple Client Applications

The system demonstrates how a single backend can support different client technologies.

Realtime Communication

The WebSocket implementation provides a foundation for realtime communication beyond ordinary REST operations.

Polyglot Persistence

Oracle, MongoDB, and Redis are used for different data requirements rather than forcing all data into one persistence technology.

9.4 Future Work

Although the current implementation provides the required foundation, several areas could be developed further.

9.4.1 Automated Testing

One of the most important improvements would be the introduction of a comprehensive automated test suite.

Future tests could cover:

backend services;

validators;

repositories;

REST APIs;

WebSocket communication;

authentication;

database integration;

9.4.2 Continuous Integration and Deployment

A CI/CD pipeline could be introduced to automate the development lifecycle.

A future pipeline could perform:

Git Push

↓

Build

↓

Static Analysis

↓

Unit Tests

9.4.3 Improved Security

Security could be strengthened further for production deployment.

Possible improvements include:
centralized secret management;
stronger production configuration;
additional authorization policies;
security auditing;
vulnerability scanning;
penetration testing;
improved token protection;

9.4.4 Scalability and Performance

The current architecture provides a foundation for scaling, but large-scale performance testing would be required before deploying the system to a large user base.

Future work could include:
load testing;
stress testing;
WebSocket connection benchmarking;
database performance optimization;
Redis optimization;
horizontal backend scaling;
monitoring of server resources.

9.4.5 Improved Voice and Video Communication

The current communication architecture focuses on realtime messaging and browser-based voice-message delivery.

Future versions could further develop this area by implementing a complete production-grade WebRTC communication layer.

Possible improvements include:
improved audio quality;
advanced realtime communication;
screen sharing;
network adaptation;
voice-message quality monitoring;
TURN/STUN infrastructure;

9.4.6 Push Notifications

A future version could provide push notifications for events that occur while a user is not actively viewing the application.

Potential notification events include:
new messages;
voice-message notifications;
mentions;
group activity;
important account events.

Notification support would improve the usability of the platform across both desktop and web environments.

9.4.7 Mobile Client

The current project focuses on desktop and web access.

A future version could introduce mobile clients for Android and iOS.

Because the backend already provides a centralized API and realtime communication protocol, a mobile client could reuse much of the existing backend infrastructure.

The shared contracts would also provide a useful foundation for implementing consistent communication behavior across additional platforms.

9.4.8 Improved UI and Accessibility

The user interface could be further improved through:

- a shared cross-platform design system;

- additional responsive layouts;

- improved accessibility;

- keyboard navigation;

- screen-reader support;

- more extensive usability testing;

- additional theme customization beyond the implemented light and dark themes;

- improved mobile layouts.

9.4.9 Advanced Search

Future versions could introduce advanced message and file search.

Possible search functionality includes:

- searching message content;

- filtering by user;

- filtering by date;

- filtering by attachments;

- searching within specific conversations;

- searching files by type.

For a communication platform with a large number of conversations, advanced search would significantly improve usability.

9.4.10 End-to-End Encryption

Another potential future improvement is end-to-end encryption for private communication.

Such a feature would require substantial architectural changes because messages would need to be encrypted before leaving the client and decrypted only by authorized recipients.

A future implementation would need to consider:

- key generation;

- key exchange;

- key storage;

- device management;

- group encryption;

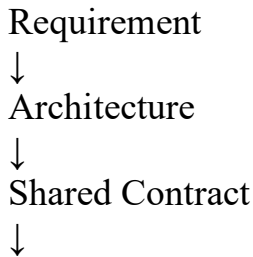
- message synchronization;

9.5 Long-Term Development

The architecture of NexusTeam provides a foundation for continued development.

The separation between clients, backend services, repositories, databases, and shared contracts makes it possible to add functionality without redesigning the entire system.

For example, a future feature can generally follow the established development process:



9.6 Final Evaluation

NexusTeam demonstrates the development of a complete software system rather than an isolated application component.

The project combines:
requirements engineering;
software architecture;
backend development;
client development;
UI design;
database technologies;
realtime communication;

9.7 Final Conclusion

NexusTeam successfully establishes a foundation for a modern multi-platform communication system.

The implementation provides the essential functionality required for user authentication, chat management, realtime messaging, attachments, user presence, organization, and additional communication features. The current voice-message functionality is provided by the web client. The combination of ASP.NET Core, web client, web technologies, WebSockets, Oracle, MongoDB, Redis, and Docker demonstrates the integration of multiple technologies into a single software system.

The modular architecture and separation of responsibilities provide a strong foundation for maintenance and future development. At the same time, the identified limitations — particularly automated testing, large-scale performance validation, production security hardening, and further development of advanced communication features — provide clear directions for future work.

The project therefore represents not only a functional communication application but also a practical demonstration of collaborative software engineering, from requirements definition through architecture, implementation, testing, project management, and documentation.

Appendix A — Diagrams



